

AUTONOMOUS AI PRODUCTION DIRECTOR

A Research and Feasibility Study for a Browser-Based Multi-Camera Production
System

Prepared by Bayarbayasgalan Enkhtulga (BeBe / Nero)
AdventureX 2026 Project Research

This document studies an AI system that can understand a live event, choose useful camera angles, control a production workflow, and automatically create edited video. The goal is not only to track a person. The goal is to build an AI director that understands what is happening and produces the story.

July 2026

Executive Summary

Professional multi-camera video normally requires several people: camera operators, a director, a vision mixer, an audio operator, and editors. This makes good production expensive and difficult for small events, schools, independent creators, community sports, panels, classrooms, and early-stage media teams. A fixed camera is cheaper, but it often produces a flat recording that does not show the speaker, reaction, screen, or action at the right time.

This research proposes an Autonomous AI Production Director. Multiple phones, webcams, IP cameras, or PTZ cameras connect to a browser-based application. Computer vision models detect and track people. Audio and visual models identify the active speaker and important events. A directing engine scores each possible shot, follows cinematic rules, and selects the next camera. The system can produce a live output and later create highlights, subtitles, chapters, and vertical social clips.

The important difference is semantic direction. Existing automatic cameras often follow a person or a ball. This project should understand why a shot is useful. If a speaker points to a presentation, the system can show the screen. If a new person starts speaking, it can choose a medium shot of that person. If the audience reacts, it can briefly show the reaction. It should also explain its decisions and allow a human operator to override them.

A complete commercial system is too large for one hackathon, but a focused prototype is realistic. The recommended prototype uses three browser camera clients, a local AI server, active-speaker detection, person tracking, a rule-based directing engine, a browser operator console, live scene switching, and automatic highlight export. A virtual PTZ crop from a wide 4K frame can demonstrate camera movement without depending on motorized hardware.

My main responsibility as the full-stack AI engineer would be to connect every layer into one reliable product: browser capture, real-time transport, backend APIs, model inference, event messages, the operator dashboard, OBS or FFmpeg control, recording, editing, deployment, and demo recovery. This integration role is important because a strong model alone is not a working production system.

Main research conclusion

The project is technically credible and visually strong if the scope stays narrow. Its best positioning is: not an auto-tracking camera, but an AI director that understands an event and produces the story.

Contents

1. Problem and Product Definition
 2. Research Background and Related Systems
 3. Proposed System Architecture
 4. Browser Application Design
 5. Computer Vision and Machine Learning
 6. AI Director and Editing Logic
 7. Full-Stack AI Engineer Role
 8. Data, Training, and Evaluation
 9. Hardware and Camera Setup
 10. Hackathon MVP and Development Plan
 11. Risks, Safety, and Ethics
 12. Product Direction and Business Potential
 13. Final Recommendation
- References

Research Questions

- Can ordinary phones and browser technology provide stable enough multi-camera input for a useful prototype?
- Can audio, vision, and directing rules choose better shots than simple person tracking?
- How should fast frame-level models and slower semantic AI work together?
- What is the smallest prototype that clearly proves the product idea at AdventureX?
- Which technical components should be owned by a full-stack AI engineer?

1. Problem and Product Definition

1.1 The production gap

High-quality video is not created by the camera alone. A human production team decides where to place cameras, which person or object matters, when to cut, how long to hold a shot, when to show context, and which moments should become highlights. Small productions usually cannot afford this team. They either record one wide shot or ask one person to control everything manually.

The problem is especially clear in panel discussions, classes, presentations, podcasts, hackathons, local sports, school events, concerts, and community performances. The content can be valuable, but the recording does not communicate it well. A useful AI production system should reduce the number of operators while keeping human control available.

1.2 Product statement

The proposed product is a browser-based control system that receives multiple live camera and audio feeds, understands the visible scene and spoken event, chooses or moves virtual cameras, produces a live program output, and creates edited deliverables after the event.

The system has five product abilities:

- Connect: turn phones, webcams, IP cameras, and PTZ cameras into a shared production session.
- Understand: detect people, faces, poses, speakers, presentation screens, actions, and reactions.
- Direct: select shots using context, quality scores, and professional editing constraints.
- Produce: send a stable live program feed with manual override and clear system status.
- Edit: export a full recording, highlights, captions, chapters, and vertical clips.

1.3 What the product is not

It is not only a face tracker, an OBS remote, a video conference, or a generative video model. These can be components, but the main product is the decision layer between raw cameras and a finished production. It should work with real captured video and make directorial choices that are visible in the final result.

1.4 Initial target scenario

The best first scenario is a three-person panel or product presentation in one room. It has clear speakers, a screen, reaction moments, and several useful camera angles. It is easier to test than a full sports match, but it still demonstrates tracking, semantics, live switching, and automatic editing.

2. Research Background and Related Systems

2.1 Intelligent cinematography

Intelligent cinematography is a research area combining computer vision, camera control, scene reconstruction, shot selection, and production rules. A 2024 review by Azzarelli, Anantrasirichai, and Bull organizes work across general, virtual, live, and aerial production. This supports the idea that automated capture and directing is a real technical field, not only a product feature [1].

Research systems also show that automatic editing improves when it considers semantics and professional directing rules. A lecture-oriented system proposed in 2024 uses lecture events to direct multiple streams in real time and edit an offline version [2]. Real Time GAZED studies online shot selection and camera trajectory optimization for virtual cameras [3]. Other work studies temporal and contextual relationships between multi-camera shots [4].

2.2 Existing commercial systems

System	Main strength	Gap for this project
Pixellot	Automated sports capture, production, replay, and highlights	Mainly sports and dedicated capture workflows
Spiideo	AI sports tracking and broadcast from installed or portable cameras	Venue and sport orientation
Vislink IQ Sports Producer	Unmanned panoramic sports production	Specialized broadcast product
AI PTZ cameras	Automatically keep a presenter in frame	Usually one-camera tracking, not semantic multi-camera direction
OBS / vMix	Strong live mixing and streaming	A human normally makes the directing decisions

Commercial evidence proves there is demand, especially in sports [5][6][7]. It also reveals the opportunity: a flexible and explainable system that works with common devices and different event types. The prototype should not claim that automatic production is new. The new contribution should be the combination of accessible browser capture, semantic event understanding, human-readable direction logic, and automatic post-production.

2.3 Technical opportunity

Modern browsers can capture camera and microphone streams, send media in real time using WebRTC, record MediaStreams, and access lower-level codec interfaces through WebCodecs [8][9]. Computer vision models can now perform detection, tracking, pose estimation, face analysis, and speech processing fast enough for a local prototype. NVIDIA also publishes multi-camera tracking workflows that maintain identities across camera views [10]. These technologies reduce the amount of custom infrastructure needed for an MVP.

3. Proposed System Architecture

3.1 Four-layer architecture

Layer	Responsibility	Example technologies
Capture	Camera permission, preview, device status, timestamps, stream upload	Browser, getUserMedia, WebRTC
Real-time platform	Sessions, signaling, stream routing, recording, event transport	Next.js, FastAPI, WebSocket, SFU, FFmpeg
AI and direction	Detection, tracking, active speaker, scoring, rules, semantic events	PyTorch, OpenCV, YOLO, Whisper
Production and editing	Scene switching, program output, virtual PTZ, highlights, subtitles	OBS, GStreamer, FFmpeg

3.2 End-to-end flow

A practical event flow is:

- The operator creates a production session in the web dashboard.

- Each phone opens a join link or scans a QR code, selects a camera, and starts a preview.
- The server receives synchronized low-latency streams and stores a higher-quality recording when possible.
- The inference service samples frames and audio, then returns detections, tracks, speaker probabilities, and quality measurements.
- The director engine combines those signals with timing rules and selects a camera or virtual crop.
- The production adapter changes the OBS scene or FFmpeg/GStreamer composition.
- The dashboard displays preview, program output, current decision, confidence, and manual controls.
- After recording, the editor service uses timestamps and semantic markers to create deliverables.

3.3 Separate live and recording paths

A major engineering decision is to separate the low-latency analysis path from the high-quality recording path. AI does not need every full-resolution frame. A 640-pixel preview sampled at 5 to 15 frames per second may be enough for detection and shot decisions. The final recording should remain at the highest stable quality. This reduces GPU, CPU, and network load.

Recommended logical paths:

Live preview -> downscale -> inference -> director decision

Original stream -> synchronized recording -> final edit

3.4 Event model

All AI results should become timestamped events instead of direct UI actions. Example event types are PERSON_ENTERED, SPEAKER_CHANGED, SCREEN_POINTED_AT, APPLAUSE_DETECTED, SHOT_QUALITY_LOW, CAMERA_OFFLINE, and DIRECTOR_CUT. This event-driven design makes the system easier to debug, replay, evaluate, and edit later.

4. Browser Application Design

4.1 Why a browser application

A browser application reduces setup. A phone can become a camera without installing a special mobile app. The operator can use a laptop dashboard, and remote team members can inspect the session. WebRTC provides real-time media transport, while Media Capture APIs request access to cameras and microphones [8]. WebCodecs can support more direct frame processing when browser compatibility allows it [9].

4.2 Camera client

The phone or browser camera page should contain only the controls required on set:

- Session name and camera label, for example Wide, Speaker A, Speaker B, or Screen.
- Front/back camera selector, resolution, frame rate, and microphone selection.
- Local preview with safe-area guides and orientation lock.
- Network quality, dropped frames, battery level, temperature warning, and connection state.
- Start, pause, reconnect, and local backup recording.

- Optional tally indicator showing whether the camera is live, previewed, or inactive.

4.3 Operator console

The operator console is the main product interface. It should feel closer to a simple production switcher than a general analytics dashboard. The operator must understand the session in one look and take manual control immediately.

Area	Purpose
Multiview grid	Show all camera feeds with labels, delay, confidence, and track overlays
Preview / Program	Show the next proposed shot and the current live output
Director timeline	Explain detected events, cuts, confidence, and rejected alternatives
Manual switcher	Cut to a camera, lock a subject, choose wide/medium/close crop
AI controls	Auto mode, assisted mode, cut speed, minimum duration, preferred subject
Health panel	GPU load, latency, packet loss, camera battery, recording status
Export panel	Full edit, highlights, vertical clip, subtitles, and transcript

4.4 Operating modes

- Manual: the system only displays AI suggestions.
- Assisted: AI prepares the next shot, but the operator confirms the cut.
- Automatic: AI selects shots and the operator can override at any time.
- Safety: the system falls back to a stable wide shot if confidence or connectivity is low.

4.5 Browser limitations

Mobile browsers can stop or reduce work when the screen locks, the tab becomes inactive, the device overheats, or the network changes. Camera access also requires permission and a secure HTTPS context. The MVP should keep the camera page in the foreground, prevent sleep where possible, show clear reconnect states, and keep a local recording or safe wide camera as backup.

5. Computer Vision and Machine Learning

5.1 Per-camera perception

Each camera is first processed independently. A detector locates people, faces, presentation screens, microphones, and any scenario-specific objects. A tracker keeps stable local identities over time. Pose estimation provides body keypoints, while face landmarks and audio features help identify the active speaker.

A per-camera observation at time t can be represented as:

$$O(c,t) = \{\text{boxes, tracks, faces, poses, audio, quality, timestamp}\}$$

5.2 Detection and tracking

A small YOLO model is suitable for a hackathon because it is fast, well supported, and easy to deploy. ByteTrack or BoT-SORT can associate detections across frames. The tracker should not decide the live shot by itself. It only provides stable subjects and movement history to the director.

5.3 Cross-camera identity

Local track IDs are different in every camera. Multi-target multi-camera tracking tries to connect them into a global identity. Appearance embeddings, face similarity, time, location, and camera topology can be combined. This is useful when the same speaker appears in a wide and close camera.

$$S(i,j) = a * S_{\text{appearance}} + b * S_{\text{face}} + c * S_{\text{space-time}}$$

For the first prototype, exact global identity is not always required. Cameras can be assigned to known subjects, or the operator can label a track. This reduces research risk while preserving the product demo. A learned re-identification module can be added later.

5.4 Active-speaker detection

Audio level alone cannot identify the visible speaker because every camera may hear the same room audio. A better approach combines voice activity detection, lip motion, face tracks, and optionally microphone direction or speaker diarization. The output should be a probability for each visible person, not only a hard label.

$$P(\text{active speaker} \mid \text{face motion, audio, track history})$$

Research in audio-visual speech recognition shows why active-speaker selection matters when several candidate faces are visible [11]. For the MVP, a practical hybrid can use Silero VAD, face/lip movement, and a short temporal smoothing window. A stronger model such as TalkNet can be tested if compute and integration time allow.

5.5 Pose, gaze, and gesture

Pose estimation helps identify pointing, hand raising, stage movement, and body orientation. A speaker facing a screen and extending an arm toward it is a useful signal for selecting the screen camera. Head pose or gaze can also indicate which person or object is receiving attention. These signals should be treated as evidence, not absolute truth.

5.6 Visual quality model

Every candidate shot should receive quality measurements:

- Subject visibility and amount of occlusion.
- Sharpness, exposure, noise, and motion blur.
- Headroom, lead room, subject scale, and centrality.
- Whether the subject is entering or leaving the frame.
- Whether another camera provides a cleaner version of the same subject.

A learned aesthetic model is possible, but clear rules are easier to debug during a hackathon. A hybrid system can use computer vision measurements for quality and a small model for more subjective composition scoring.

5.7 Speech and semantic understanding

Speech-to-text provides semantic context. Keywords and sentence meaning can indicate a demo reveal, conclusion, question, transition, or reference to a slide. A slower language or vision-language model can process short windows, while the fast director continues using tracking and audio. The semantic model should produce structured events, not directly control the camera.

6. AI Director and Editing Logic

6.1 Candidate shot generation

The director does not only choose a physical camera. It chooses a candidate shot. One wide camera can generate a wide shot, a medium crop, or a close crop. A candidate contains the source camera, crop, subject, shot size, quality, and semantic purpose.

6.2 Shot score

A simple interpretable score is:

$$Q(k, t) = w_v * V + w_a * A + w_c * C + w_s * S + w_n * N - w_p * P$$

- V: visibility of the important subject.
- A: active-speaker relevance.
- C: composition and technical quality.
- S: semantic match to the current event.
- N: novelty compared with recent shots.
- P: penalties for blur, occlusion, low confidence, or unstable motion.

6.3 Temporal rules

Selecting the highest score every frame will create rapid and unpleasant cuts. The director therefore needs temporal rules. It can require a minimum shot duration, use a cut cooldown, prefer the current shot when scores are close, and limit repeated transitions. A strong reason, such as a confirmed speaker change, can override the normal hold time.

Choose new shot only if:

$$Q_{\text{new}} - Q_{\text{current}} > \text{threshold AND } \text{duration}_{\text{current}} > \text{minimum}$$

6.4 Directing grammar

Event	Preferred response
New speaker	Medium or close shot after stable confirmation
Speaker points to screen	Screen shot or picture-in-picture
Question from another person	Questioner, then response speaker
Strong reaction or applause	Short reaction shot, then return to main action
Low confidence	Hold current valid shot or fall back to wide
Long unchanged shot	Select a contextually valid alternative
Camera failure	Remove candidate immediately and use safe source

6.5 Explainability

The dashboard should show short explanations such as: 'Camera 2 selected: new active speaker, clean medium framing, confidence 0.86.' This is useful for trust, debugging, and judging. It also makes the AI contribution visible during the demo instead of hiding it behind automatic cuts.

6.6 Automatic post-production

The same timestamped events can drive offline editing. Offline processing has more time and can use higher-quality models. It can correct a bad live cut, use the original camera recordings, improve synchronization, and create several output formats.

A highlight score can combine:

$$H(t) = a * \text{audio excitement} + b * \text{motion} + c * \text{reaction} + d * \text{semantic importance}$$

- Full edited recording with corrected camera selection.
- Thirty- to sixty-second highlight reel.
- Nine-by-sixteen social clip with subject-aware reframing.
- Transcript, subtitles, chapter titles, and speaker labels.
- Suggested thumbnail frames and key moments.

7. Full-Stack AI Engineer Role

7.1 Role definition

In this project, the full-stack AI engineer is not only a web developer and not only an ML engineer. The role owns the path from a real camera to a real production output. My responsibility would be to make the models, backend, browser interface, video tools, and deployment work as one product.

7.2 Main responsibilities

Area	My responsibility
Browser capture	Build the phone camera client, permissions, previews, connection status, and session joining
Real-time transport	Connect WebRTC/signaling, timestamps, WebSocket events, stream health, and reconnection
Backend	Design session, camera, event, decision, recording, and export APIs
AI integration	Wrap models as services, normalize outputs, batch inference, and manage GPU/CPU resources
Director engine	Implement scoring, rules, state transitions, confidence, and decision explanations
Production control	Connect decisions to OBS, FFmpeg, or GStreamer and maintain a safe fallback output
Editing pipeline	Use recordings and event timestamps to build cuts, subtitles, highlights, and vertical outputs
Operator UI	Build multiview, Preview/Program, timeline, manual override, status, and export controls
Reliability	Logging, health checks, reconnect logic, degraded modes, test recordings, and demo recovery
Deployment	Package services, configure devices, measure latency, and run the full system locally

7.3 AI engineering work

AI engineering means converting research models into a stable real-time service. I would define input and output schemas, choose frame rates and resolutions, measure inference time, combine model results, cache tracks, smooth predictions, handle missing data, and expose clear confidence values. I would also decide which work runs in the browser, backend, GPU service, or offline editor.

7.4 Full-stack work

The full-stack part includes the user journey and the system state. A camera needs to join a session, receive a label, stream media, reconnect, and report health. The operator needs immediate previews, decisions, controls, and recording status. The editor needs access to synchronized files and event timelines. These workflows require database design, APIs, real-time messages, frontend state, authentication or session tokens, and reliable error handling.

7.5 Integration ownership

The most important part of my role is integration ownership. A detection model can work in a notebook while the browser stream is delayed, OBS is on the wrong scene, and the exported video has unsynchronized audio. I would own the complete demo path and make trade-offs so the final product works. This matches my strongest skill: building quickly and connecting backend, frontend, APIs, AI services, dashboards, and automation into a working MVP.

7.6 Collaboration with the team

- Jeboyn leads product direction, UI/UX, visual language, demo flow, storytelling, and pitch.
- Ceaser can lead deeper ML work, optimization, deployment, tracking, and hardware or systems tasks.
- I lead full-stack implementation, AI integration, production control, dashboard, and end-to-end reliability.
- Shared work includes directing rules, dataset recording, testing, evaluation, and the final demonstration.

8. Data, Training, and Evaluation

8.1 Start without custom training

The first prototype should use pretrained models and engineering rules. Training a complete directing model requires a large aligned dataset of multiple camera feeds, audio, shot decisions, and event labels. That is not realistic before the core pipeline works. Pretrained detection, tracking, pose, speech, and embedding models are enough to test the product hypothesis.

8.2 Small project dataset

The team can record a small controlled dataset with three cameras and several scripted situations:

- One speaker, speaker change, interruption, and overlapping speech.
- Pointing to a display, product reveal, and live demonstration.
- Audience question, laughter, applause, and silence.
- Occlusion, bad framing, subject leaving frame, and temporary camera failure.

- Different lighting, distances, backgrounds, clothing, and phone models.

For each session, a human can create a reference edit and annotate speaker changes, gestures, important moments, and acceptable shots. This becomes an evaluation set and later a source of training examples.

8.3 Evaluation metrics

Component	Metric
Detection	Precision, recall, and missed important subjects
Tracking	ID switches, track continuity, and recovery time
Active speaker	Frame accuracy, F1, and speaker-change delay
Direction	Agreement with human edit, invalid cut rate, shot-duration quality
System	End-to-end decision latency, dropped frames, reconnect time
Editing	Highlight precision, subtitle quality, audio/video synchronization
Human evaluation	Clarity, comfort, usefulness, trust, and preference

8.4 Latency budget

The system does not require every decision to be instant. A one- to two-second confirmation can produce a more stable speaker cut than reacting to every sound. Still, the pipeline should measure each stage. A target prototype budget could be 100-250 ms for transport, 50-150 ms for visual inference, 100-300 ms for audio evidence, and less than 50 ms for the rule engine. Semantic events can arrive later and update the next decision.

8.5 Comparison baseline

The prototype should be compared against:

- A fixed wide camera.
- A simple rule that always selects the loudest microphone or largest face.
- The proposed multimodal director.
- A short human-edited reference.

A small blind preference test can show participants the four outputs without labels. Even ten to twenty ratings can provide useful evidence for a hackathon presentation, as long as the limitations are stated honestly.

9. Hardware and Camera Setup

9.1 Recommended prototype setup

Item	Purpose	Minimum option
Three cameras	Wide, speaker/panel, screen or reaction angles	Phones or USB webcams
Operator laptop	Dashboard, composition, recording, and editing	Modern CPU, preferably NVIDIA GPU
Local network	Stable camera transport	Dedicated Wi-Fi router or wired adapters
Audio source	Cleaner speech evidence	USB microphone or audio interface
Tripods / mounts	Stable framing	Basic phone tripods
Power	Avoid battery failure	Chargers and power banks

9.2 Camera layout

- Camera A: safe wide shot showing the full scene and acting as fallback.
- Camera B: medium or close view of the primary speaker area.
- Camera C: second speaker, audience reaction, product table, or presentation screen.

A wide 4K source is especially useful because software can crop several virtual shots from it. This virtual PTZ method is safer than physical camera movement and still demonstrates AI framing. Resolution is traded for flexibility, so the final output may be 1080p or 720p.

9.3 Physical PTZ extension

A motorized PTZ camera can be controlled through ONVIF, VISCA over IP, vendor APIs, or a custom pan-tilt mount. The target offset from the desired composition can drive a proportional or PID controller. Dead zones, speed limits, motion prediction, and smooth acceleration are required to avoid visible shaking.

$$ex = x_{target} - x_{desired}; ey = y_{target} - y_{desired}$$

Physical PTZ should be treated as an extension, not the foundation of the hackathon demo. The core product value is direction and production intelligence. A virtual crop proves that value with less mechanical risk.

9.4 Network planning

Three high-bitrate phone streams can overload a shared event Wi-Fi network. A dedicated local router, reduced preview bitrates, adaptive resolution, and local recording are recommended. The system should show network health and automatically reduce analysis quality before it loses the program output.

10. Hackathon MVP and Development Plan

10.1 Core demonstration

The recommended demo is a live three-person panel or product presentation. The system automatically selects the active speaker, shows the screen when it is referenced, uses a reaction shot for applause or laughter, and returns to a safe wide shot when confidence is low. The operator can override any decision. At the end, the system creates a short highlight and vertical clip.

10.2 Must-have scope

- Three browser camera clients with stable joining and preview.
- Synchronized multiview and one safe recording path.
- Person detection/tracking and practical active-speaker logic.
- Interpretable shot scores, minimum duration, cooldown, and fallback.
- Operator console with Preview, Program, AI explanation, and manual cut.
- OBS or FFmpeg scene control and recorded output.
- Automatic transcript and one short highlight export.

10.3 Nice-to-have scope

- Virtual PTZ with smooth subject-aware crop.
- Gesture-to-screen event and picture-in-picture.
- Cross-camera re-identification.
- Vertical reframe and automatic subtitle styling.
- Remote viewer or stream output.
- Physical PTZ or custom hardware controller.

10.4 Suggested 120-hour sequence

Phase	Focus	Exit condition
0-12 h	Lock demo story, camera layout, interfaces, and fallback	Recorded manual three-camera path works
12-30 h	Browser capture, session backend, multiview, timestamps	Three clients stay connected
30-50 h	Detection, tracking, audio/VAD, event schema	Live structured observations appear
50-70 h	Director state machine, scores, cut rules, OBS/FFmpeg	Automatic stable camera switching
70-88 h	Operator UI, explanations, manual override, health	A user can operate and recover the system
88-102 h	Recording, transcript, highlight, vertical export	One complete session exports
102-112 h	Testing, latency tuning, failure cases, backup demo	Repeatable demo succeeds several times
112-120 h	Pitch, video, metrics, final polish	Team can present without live risk

10.5 Demo recovery plan

A hackathon demo should have three levels: a live demo, a local pre-recorded session processed by the same system, and a short final video showing the intended result. If Wi-Fi or a camera fails, the team can still demonstrate the director timeline and replay the recorded feeds through the system. This is not fake; it is a controlled fallback for unreliable event conditions.

11. Risks, Safety, and Ethics

11.1 Technical risks

Risk	Mitigation
Different stream delays	Timestamp buffers, clap/audio alignment, and measured offset correction
Unstable Wi-Fi	Dedicated router, adaptive bitrate, local recording, and safe wide camera
Speaker mistakes	Temporal smoothing, confidence threshold, operator override
Rapid cuts	Minimum duration, hysteresis, cooldown, and transition penalties
GPU overload	Lower analysis resolution/FPS, model batching, and staged semantic processing
Camera/browser stops	Visible health status, auto reconnect, wake lock, and fallback
Scope becomes too large	Protect must-have features and remove physical PTZ first

11.2 Privacy and consent

The system records people, faces, voices, and possibly audience reactions. Participants should know when recording and AI analysis are active. The prototype should use clear consent, visible recording status, limited data retention, and local processing where possible. Face recognition should not be required for the MVP. Track labels or temporary embeddings are safer than storing permanent identities.

11.3 Bias and creative control

An AI director can repeatedly select the loudest, most central, or most visually clear person and ignore others. That can produce unfair representation. The operator should be able to lock, exclude, or balance subjects. The decision timeline should reveal how often each person appears. Automatic editing should support creative workers, not hide or remove their contribution.

11.4 Security

Camera sessions need unguessable join tokens, transport encryption, limited permissions, and session expiration. Production controls should not be open to every connected camera. Recordings and transcripts should have access control. For the hackathon, the system can stay on a local trusted network while still using signed session tokens and a clear separation between camera and operator roles.

12. Product Direction and Business Potential

12.1 Beachhead markets

The long-term product should not target every kind of production at the beginning. The strongest first markets are situations with repeated layouts, limited budgets, and clear value from automation.

Market	Why it fits
Panels and conferences	Clear speakers, screens, repeated event grammar, expensive crews
Schools and lectures	Many rooms, limited staff, need for recordings and chapters
Podcasts and interviews	Speaker switching, social clips, regular production
Community sports	Strong demand, but requires sport-specific tracking later

Market	Why it fits
Houses of worship	Repeated setup, speakers, music, and volunteer operators
Creator studios	Fast production, highlights, vertical content, and remote control

12.2 Possible business models

- Subscription per production room or venue.
- Pay per recorded hour, live event, or export.
- Software license for production companies and schools.
- Hardware kit containing cameras, mounts, router, and local AI computer.
- API or SDK for existing camera and streaming platforms.

12.3 Defensibility

The strongest defensibility is not a single detector. Many detection and speech models are available. Value can come from a growing multi-camera event dataset, directing policies, production integrations, reliable edge deployment, human feedback, and domain-specific event grammars. The product should learn from operator corrections without making the operator lose control.

12.4 Product positioning

A simple message is more effective than a long feature list: Connect your cameras. The AI understands the event, directs the live video, and edits the final story. For AdventureX, the live visual proof is more important than claiming a complete business. The team should show one narrow scenario working very well and explain how the same architecture can extend to other domains.

13. Final Recommendation

The Autonomous AI Production Director is a strong AdventureX direction because it combines computer vision, audio intelligence, real-time systems, browser technology, product design, and visible creative output. It is more memorable than a normal web application, but it can still be prototyped mostly in software with ordinary cameras.

The team should avoid building a universal production platform in 120 hours. The project will be stronger if it proves one complete loop: cameras connect, the AI understands a panel or presentation, it selects stable shots, a human can override it, and the system exports a useful edited clip. Physical camera robotics and advanced learned directing can remain future work.

My role as the full-stack AI engineer is central to this plan. I would turn separate technical ideas into one working system, including real-time camera ingestion, APIs, AI service integration, the director engine, browser controls, production output, editing automation, and deployment. Jeboy's product and creative direction, Ceaser's deeper ML and systems work, and my integration and MVP building can form a balanced team.

The final project should be presented honestly. Existing automatic production products already prove the market. Our goal is not to pretend the category is new. Our contribution is to make the director more accessible, semantic, explainable, and connected to both live and post-production workflows.

Final concept statement:

We are building an AI director for real cameras. It watches every angle, listens to the event, understands who and what matters, chooses the next shot, keeps a human in control, and turns the session into finished content.

References

- [1] A. Azzarelli, N. Anantrasirichai, and D. R. Bull. Intelligent Cinematography: A Review of AI Research for Cinematographic Production. Artificial Intelligence Review, 2024. <https://arxiv.org/abs/2405.05039>
- [2] A Multi-Purpose Automatic Editing System Based on Lecture Semantics for Real-Time Broadcasting and Offline Editing, 2024. <https://arxiv.org/abs/2411.04859>
- [3] Real Time GAZED: Online Shot Selection and Editing of Virtual Cameras for Live Production. WACV 2024. <https://arxiv.org/abs/2311.15581>
- [4] Temporal and Contextual Transformer for Multi-Camera Editing of TV Shows, 2022. <https://arxiv.org/abs/2210.08737>
- [5] Pixellot. Automated Sports Filming, Production and Broadcasting. <https://www.pixellot.tv/solutions/produce/>
- [6] Spiideo. AI Sports Video Camera Systems. <https://www.spiideo.com/sports-video-camera-systems/>
- [7] Vislink. IQ Sports Producer. <https://www.vislink.com/product/iq-sports-producer/>
- [8] MDN Web Docs. WebRTC API; Media Capture and Streams API. https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API
- [9] World Wide Web Consortium. WebCodecs Specification. <https://www.w3.org/TR/webcodecs/>
- [10] NVIDIA. Multi-Camera Tracking AI Workflow and DeepStream SDK. <https://www.nvidia.com/en-us/ai-data-science/ai-workflows/multi-camera-tracking/>
- [11] A Closer Look at Audio-Visual Multi-Person Speech Recognition and Active Speaker Selection, 2022. <https://arxiv.org/abs/2205.05684>
- [12] OBS Studio Documentation. Core, Media, Scripting, and Production APIs. <https://docs.obsproject.com/>
- [13] Automatic Camera Control and Directing with an Ultra-High-Definition Panoramic Camera, 2022. <https://arxiv.org/abs/2208.05213>
- [14] Context-Based Camera Selection from Multiple Video Streams. Multimedia Tools and Applications, 2021. <https://doi.org/10.1007/s11042-021-11674-6>
- [15] NVIDIA DeepStream Open Source Repository. Multi-stream, inference, tracking, and media pipeline references. <https://github.com/NVIDIA/DeepStream>

Appendix A. Suggested Initial API Objects

The following objects keep the implementation clear without defining a final commercial API.

Session: id, title, mode, state, created_at, started_at, stopped_at

Camera: id, session_id, label, role, stream_state, latency_ms, health

Observation: timestamp, camera_id, tracks, faces, poses, audio, quality

Event: timestamp, type, subject_id, source, confidence, payload

ShotCandidate: source, crop, subject, shot_size, score, reason

Decision: timestamp, selected_shot, previous_shot, confidence, explanation

Export: id, session_id, type, status, file_path, duration, metadata

Appendix B. Recommended Technology Stack

Part	Recommended MVP choice	Alternative
Web UI	Next.js / React / TypeScript	Vue or plain React
API	FastAPI / Python	Node.js service for signaling
Real-time events	WebSocket	Socket.IO
Media transport	WebRTC	SRT or RTSP for controlled devices
Video processing	FFmpeg and OpenCV	GStreamer / DeepStream
Detection	Small YOLO model	RT-DETR
Tracking	ByteTrack or BoT-SORT	NVIDIA NvDCF
Pose	MediaPipe or YOLO Pose	RTMPose
Speech	Whisper plus Silero VAD	Cloud transcription API
Production	OBS plus WebSocket/script	Native GStreamer composition
Storage	SQLite for MVP	PostgreSQL and object storage
Packaging	Docker Compose where practical	Direct local processes

End of document.